计算机组成原理 CPU 项目报告

黄政东 12212230

祝超 12210305

董万希 12210432

1 开发说明

1.1 分工与贡献比

姓名	学号	负责内容	贡献比
黄政东	12212230	cpu 结构设计与编写，single cycle 与 pipeline 设计与编写，uart, 中断指令，编译链小工具，文档写作	$\frac{1}{3}$
祝超	12210305	cpu 部分基本结构，cpu 结构功能测试，基本指令测试，场景 1 2 汇编测试，文档写作	$\frac{1}{3}$
董万希	12210432	cpu 部分基本结构，场景 2 汇编测试，7 段数码管，vga，文档写作	$\frac{1}{3}$

表 1: 开发者说明

1.2 开发日程

05.06 搭建 chisel 开发环境

05.08 完成 cpu single cycle 版本的结构设计

05.10 实现 cpu 各基础模块内部结构

05.12 完成基础模块功能测试，实现自定义编译链工具

05.13 完成场景 1 汇编指令编写

05.15 完成中断控制硬件设计

05.16 中断控制硬件实现

05.18 完成场景 2 汇编指令编写

05.19 汇编场景测试与 debug

05.21 汇编实现测试中断程序

05.22 完成 pipeline 设计

05.24 完成 pipeline 实现

05.25 pipeline 测试

05.26 cpu 完整功能测试

2 CPU 架构设计

2.1 CPU 特性

2.1.1 ISA 实现

ISA 指令集的实现参考于 RISC 精简指令系统计算机设计结构，通过对指令 32bits 的分解实现对指令种类的分析与传递路径的规划。基础指令共分为 5 类：R-type, I-type, S-type, B-type, U-type, J-type，核心指令格式如下：

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

图 1: 核心指令格式

在此基础上我们进一步实现了 zicsr 指令集以及与中断处理相关的 ecall,ebreak,mret 三条指令，zicsr 指令的格式如下，其功能将在 bonus 章节具体阐述

31 20	19 15	14 12	11 7	6 0
csr	rs1	funct3	rd	opcode
12	5	3	5	7
source/dest	source	CSRRW	dest	SYSTEM
source/dest	source	CSRRS	dest	SYSTEM
source/dest	source	CSRRC	dest	SYSTEM
source/dest	uimm[4:0]	CSRRWI	dest	SYSTEM
source/dest	uimm[4:0]	CSRRSI	dest	SYSTEM
source/dest	uimm[4:0]	CSRRCI	dest	SYSTEM

图 2: Enter Caption

2.1.2 CPU 性能

我们实现了多周期五级流水 CPU，采用分别采用分支预测与数据前递的方式解决了控制冒险与数据冒险。该系统的每条指令的平均 CPI 约为 3，在 minisys 开发板上可以达到 60MHz 的主频。

2.1.3 寻址空间设计

本项目的 CPU 系统属于哈佛架构，指令区内存与数据区内存大小分别为 64KiB，具体各空间的地址范围如图3所示

2.1.4 外设 IO 支持方式

采用 MMIO 外设，通过软硬件协同操作的方式实现与外设输入输出的交互。对应寻址范围自 0xffff_ff00 起，不同设备对应末 4bit 不同。本项目目前采用轮询方式

IO 设备	起始地址末 4bit
led 24bit	0x0
button 5bit	0x4
switch 24bit	0x8
7 段数码管 32bit, vga 32bit	0xC

表 2: IO 地址

实现对开发板上 IO 设备的访问，但同时在 CPU 架构中预留了 PLIC 模块接口与对

含义	基地址/长度
指令段内存基地址	0x0000_0000
指令段内存长度	0x0000_ffff
数据段内存基地址	0x0001_0000
数据段内存长度	0x0000_ffff
.data段基地址	0x0001_0000
.bss段基地址（可自定义）	0x0001_0400
.heap段基地址（可自定义）	0x0001_0400
global pointer地址（可自定义）	0x0001_0200
stack基地址	0x0001_ffff
mmio范围基地址	0xffff_ff00

图 3: 各空间地址范围表

外部中断的处理机制，可以扩展支持外部 IO 设备触发中断的交互方式。

2.2 CPU 接口

整体 CPU 时钟源于 clock wizard IP 核分频所得到的时钟，该 IP 核的输入时钟位 Minisys 开发板内部的 100MHz 时钟晶振，引脚为 Y18。而复位信号均为高电平有效，连接至开发板上的 P20 引脚。在 UART 设备上我们使用了一个轻量级的开源 IP 核 [4]，并在此基础上进行了封装以用于初始数据的导入，其对应的 rx 引脚为 Y19，tx 引脚为 V18

2.3 CPU 内部结构图

整体系统可以分为 CoreTop 与 DeviceTop 两个模块，分别负责 CPU 整体的功能运行与开发板外设的信号处理，系统顶层架构图如图4所示

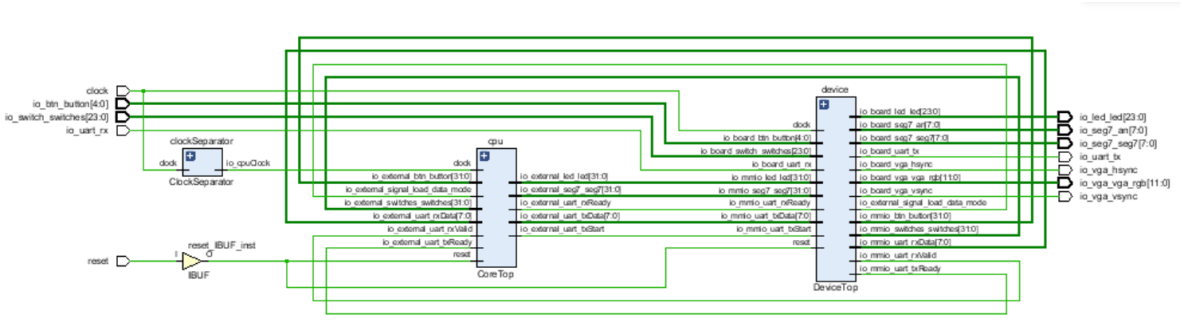


图 4: 系统顶层架构图

而 CPU 内部模块与各模块之间的连线关系如图5所示

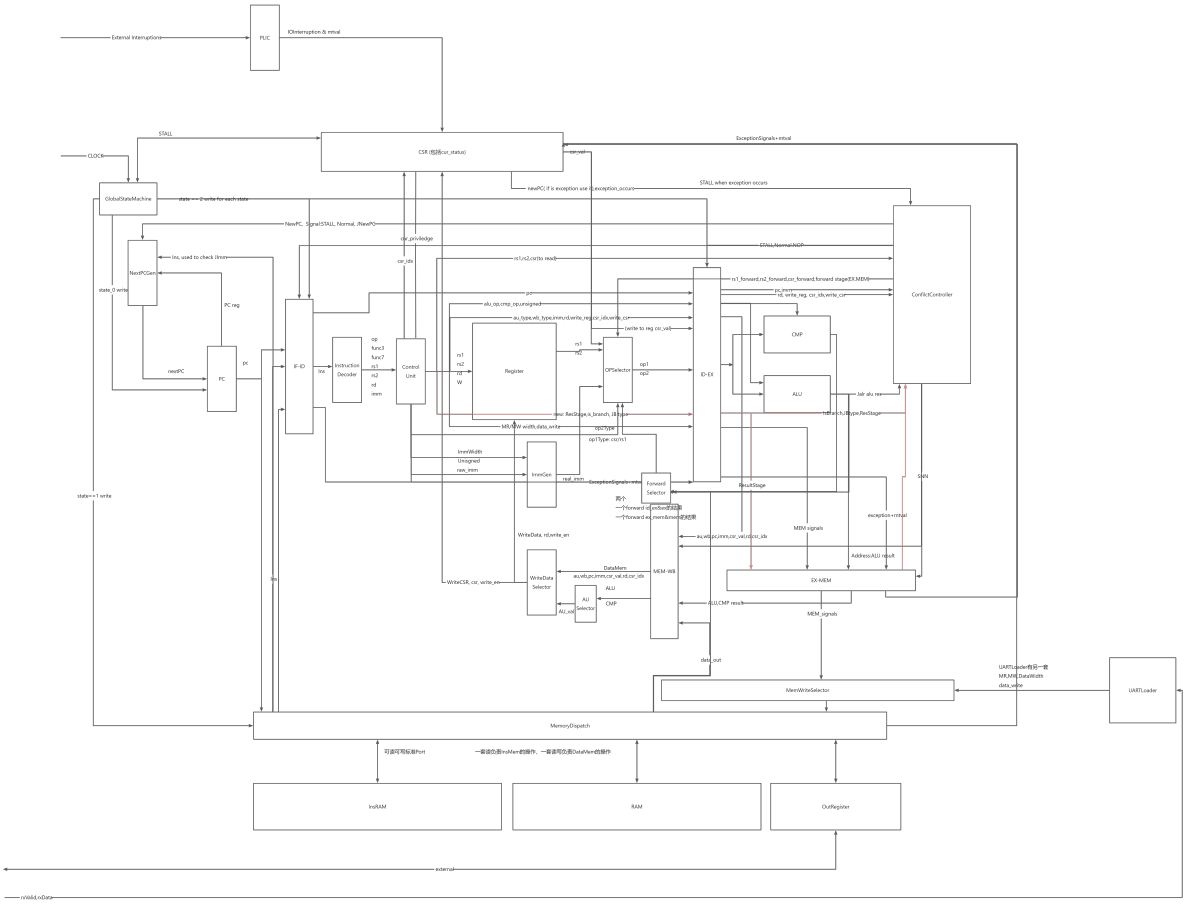


图 5: CPU 整体架构图

各阶段子模块功能与主要接口如图??所示

阶段	模块	功能	输入源	输入信号	输出信号
取指	NextPCGen	生成下一次取值的地址	ConflictController PC	ConflictControlSignal, PredictionResult, prev_pc	next_pc
	PC	存储当前指令地址	NextPCGen	next_pc	address
	InsMem	存储并取用指令	PC reg, MemoryDispatch	address, 控制信号	Instruction
	IF-ID	存储取指阶段的生成信号提供给下一模块	PC，InsMem	address, Instruction 等信号	同输入信号
	InsDecoder	对指令进行初步解码	IF-ID	Instruction	opcode,func3,func7,rs1,rs2,rd,imm
	ControlUnit	对指令进行进一步解码	InsDecoder	初步解码信号	各模块的控制信号，如 Operand1Type，Operand2Type，AUSelector等

图 6: 取址阶段

3 测试场景汇编实现说明

这里我们罗列了一些我们认为汇编实现中比较关键的部分，对其实现方式加以说明。

3.1 实现与硬件的交互

我们在硬件上实现了 MMIO 以及 ecall 两套交互逻辑，但是我们的测试场景普遍基于 MMIO 实现，ecall 部分将在 bonus 部分详细叙述。对于通过 MMIO 实现的硬件交互，我们首先约定了硬件的地址（具体见表2），然后汇编代码上对应的操作就是去读取那一块内存的值（从拨码或者按钮读）或者把某一个寄存器的值写入那一片地址（点亮 led 灯，七段数码管或者 vga）。特别地，由于读入的时候可能会存在汇编代码需要“停下来”等待对应输入的情况，我们设置了一个确认位，只有当确认键被激活的时候汇编代码才会读出对应的值，否则它就只会在一个小循环里等待确认件的输入。

译码	InsDecoder	对指令进行初步解码	IF-ID	Instruction	opcode,func3,func7,rs1,rs2,rd,imm
	ControlUnit	对指令进行进一步解码	InsDecoder	初步解码信号	各模块的控制信号，如Operand1Type，Operand2Type，AUSelector等
	Regs	存储整数指令集的32个寄存器	CU WriteDataSelector	rs1，rs2（CU） rd，w-enable（WBSelector）	rs1,rs2
	RegsForwording	依据控制信号选取实际的reg值	Regs, EXWBSelector, MEMWBSelector	ControlSignal RegsResult, EXResult,MEMResult	真实寄存器值
	ImmGen	偏移生成立即数	CU	ImmType, raw_imm	real_imm
	OPSelector	选择是使用寄存器值，CSR寄存器值还是立即数	RegsForwarding,ImmGen	Operand1Type,Operand2Type, ImmGen	operand1,operand2
	ID-EX	存储这一阶段的信号	OPSelector, IF-ID, CU	这一阶段与上一阶段的生成信号，如op1,op2,address等	同输入信号

图 7: 译码阶段

执行	ALU	运算单元	ID-EX	op1,op2,operator,unsigned	alu_result
	CMP	比较单元	ID-EX	op1,op2,operator,unsigned	cmp_result
	EXWBSelector	EX阶段写回	ALU,CMP,ID-EX	op1,op2等	exwbresult
	EX-MEM	存储这一阶段的信号	ALU,CMP, ID-EX	alu_result,cmp_result,先前阶段的必要信号	同输入信号
访存	DataMem	数据段内存	MemoryDispatch	address, data, write_en	data
	OutRegisters	外部IO寄存器	MemoryDispatch	address, data, write_en	data
	MemoryDispatch	内存映射管理	EX-MEM	address, data, write_en	data
	UARTLoader	UART功能加载单元	外部UART单元	外部IO（data,data_valid)	data,address,w_en
	MEMWBSelector	MEM阶段写回	EX-MEM	au_result, data	memwbresult
	MEM-WB	存储这一阶段的信号	MemoryDispatch, EX-MEM	data，先前阶段的必要信号	同输入信号
写回	WriteDataSelector	片选写回的数据	MEM-WB	WBType, data,result等	write_en, write_data

图 8: 执行、访存与写回

冒险控制	ConflictController	冲突控制	ID模块,IF-ID,ID-EX,EX-MEM,MEM-WB	regs_to_write, regs_to_read, instruction_type等	IF-ID,ID-EX,EX-MEM,MEM-WB,NextPCGen控制信号
中断控制	CSR	CSR寄存器，收集中断信号并处理	InsMem,DataMem,CU	IFFault, IDFault, MEMFault	cur_privilege, interruption_occurs
	PLIC	接收外部中断	外部IO	CSR	interruptions

图 9: 流水线冒险控制与中断控制

3.2 实现测试用例的转换

在两个测试场景内的测试用例的转换方面，我们的实现逻辑都是一样的，即在退出某个测试用例之后，重新回到测试用例选择模块，并保留上一次测试用例的结果（也即不会立刻清空上一次测试的输出），等到确认键再次被按下，汇编程序进入到新一轮的测试用例以后，我们会立刻将七段数码管，vga 以及 led 灯等输出设备恢复成默认状态，以便显示新一次测试的结果。在选择测试用例时候，事实上我们使用的就是一个轮询操作，找到与输入的测试用例匹配的输入后，就跳转到对应的测试用例的汇编代码处即可。

3.3 实现特殊信号输出

由于在浮点数转换样例中，可能涉及 INF,NaN 等情况，我们在硬件上同时设计了几个特殊的七段数码管输出。对此，我们约定，如果给到七段数码管的最高 2bit 分别为 01，10，11 而其他位均为 0 的时候，七段数码管上并不输出其对应的 16 进制数，而是输出 error，inf 以及 NaN。由于其它的测试用例不可能有类似的输出（因为太大了），因此我们在这里使用了这三个值。

3.4 实现信号输出停留 1-2 秒

在无需输入的情况下，我们的代码可以实现对栈内的数值进行自动展示，每个展示 1-2s，实际上是我们在汇编代码内部维护了一个 counter，而这个 counter 在跑一段循环，每当循环执行了一轮，counter 就 +1，而每当 counter 达到一个固定值 x（x 用于控制停留时长），程序就会将 counter 清零并展示下一个数，如此循环往复，直到所

有我们需要展示的数都展示完毕或者外部通过输入强行中断为止。

4 上板使用说明

在开发板方面，我们使用的是 Minisys 实验板，该开发板是一个以 Xilinx Artix-7TM 系列 FPGA（XC7A100T FGG484C-1）为主芯片的实验平台。我们实现的 CPU 以及对应的汇编代码使用开发板上各部分外设的情况如图所示：

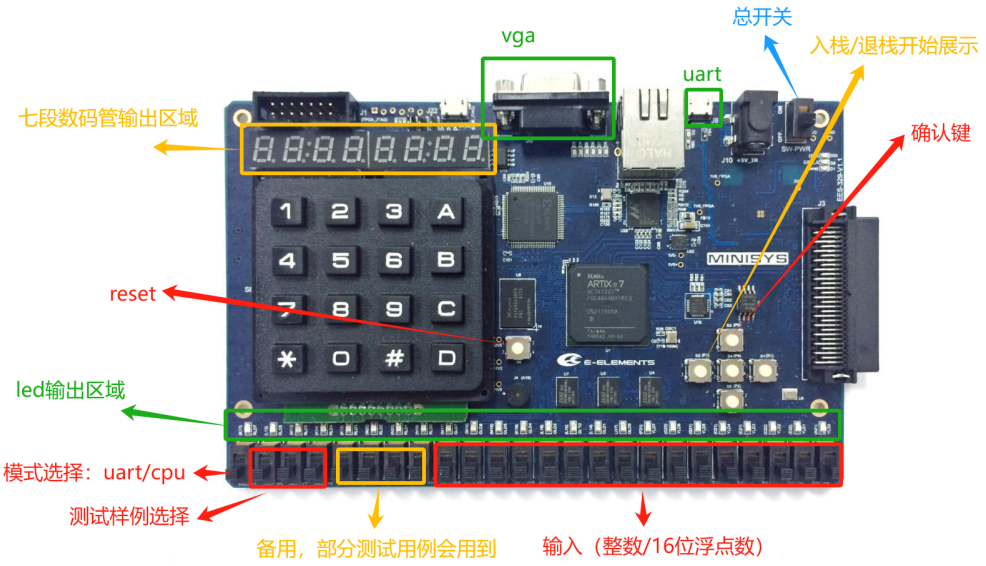


图 10: minisys 开发板各部分功能

5 Bonus 实现说明

5.1 lui 与 auipc 指令实现说明

在指令集的实现上，我们参考了 RISC-V 指令集手册 (2019) [1] 完整实现了 rv32i 基本整数指令集。对于 lui 与 auipc 两条指令，这两条指令涉及取用 PC 值并写回寄存器，我们对其的分析与建模如下：

指令	执行(ALU-OP)	访存	写回	PC更新
lui	NA	NA	rd=immGen<<12	PC+4
auipc	NA	NA	rd=PC+immGen<<12	PC+4

图 11: 对 lui 指令与 auipc 指令的分析

在该指令的实现上，我们选择在解码上述两条指令时生成 ALU add 信号，并生成信号控制 MUX, 选择 ALU 操作数 1 为 PC，而 ALU 操作数 2 的为 ImmGen 模块的输出，并生成控制信号在 ImmGen 模块完成立即数的位移，最后将 ALU 的输出写回至指定寄存器中。在上述实现中，CPU 微命令与实际指令之间并非一一对应关系，而是由译码阶段完成依据实际指令生成 CPU 各模块的控制信号。因此，该设计方式实现了 CPU 微命令与实际指令之间的解耦，通过更改译码阶段的映射逻辑，该设计方式可以较为便捷地实现对不同指令集的支持。

5.2 两模式特权架构, 中断控制硬件,ecall 与 ebreak 实现说明

对于 ecall 与 ebreak 两条指令的实现，我们参考 RISC-V 指令集手册 (2019) 特权架构卷 [2] 实现了基本的 zicsr 指令集，支持两模式 (User 模式与 Machine 模式) 的特权架构与中断控制硬件，并在其基础上编写了中断服务程序软件以提供对 ecall 与 ebreak 的支持。

5.2.1 zicsr 指令集

对于特权架构与中断服务程序的实现，需要一组内部寄存器保存 CPU 当前的运行与控制状态并协助实现模式之间的切换，这在官方手册中被描述为 CSR 寄存器 (Control and Status Registers)。在我们的实现中，我们实现了最基本的 CSR 寄存器，其列表与含义如图12

寄存器名称	寄存器编号	含义
mstatus	0x300	CPU内部状态(包括中断控制与先前运行模式等，各段含义参考官方文档说明)
mie	0x304	中断启用信号
mtvec	0x305	中断服务程序基地址
mscratch	0x340	中断服务程序的暂存寄存器
mepc	0x341	中断源地址
mcause	0x342	中断原因
mtval	0x343	中断备注信息(如bad address/instruction)
mip	0x344	记录是否有中断被挂起

图 12: 基本 CSR 寄存器说明

在此基础上，我们实现了 zicsr 所规定的六条指令，从而支持了对 CSR 寄存器的访问控制，其功能描述与控制微信号如图13，得益于微命令与实际指令之间的分离，我们能够较为便捷的实现对该功能的支持。

指令	功能	操作数	ALU命令
CSRRW	读CSR寄存器的值到一个寄存器，并用另一个寄存器的值写入CSR寄存器。	op1: x0 op2: rs1	ADD
CSRRS	读CSR寄存器的值到一个寄存器，并用另一个寄存器的值设置（置位）CSR寄存器。	op1: CSR op2: rs1	OR
CSRRC	读CSR寄存器的值到一个寄存器，并用另一个寄存器的值清除（清零）CSR寄存器。	op1: CSR op2: rs1	Not2And
CSRRWI	读CSR寄存器的值到一个寄存器，并用一个立即数写入CSR寄存器。	op1: x0 op2: uimm	ADD
CSRRSI	读CSR寄存器的值到一个寄存器，并用一个立即数设置（置位）CSR寄存器。	op1: CSR op2: uimm	OR
CSRRCI	读CSR寄存器的值到一个寄存器，并用一个立即数清除（清零）CSR寄存器。	op1: CSR op2: uimm	Not2And

图 13: zicsr 基本六条指令描述

5.2.2 两模式特权架构

由于 zicsr 指令会对 CSR 寄存器进行访问与修改操作，而这些寄存器对于中断服务等控制至关重要，因此在实际应用中，需要确保只有可信的基础程序 (如中断服务程序) 可以运行在高权限级别下执行 CSR 操作指令，而对于外部程序则不能直接对 CSR 寄存器执行操作。RISCV 官方文档定义了三种权限等级，其说明示意如图14

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	*Reserved*	
3	11	Machine	M

图 14: 权限示意图

我们基于上述文档实现了基本的两模式权限架构，支持 U 模式 (用户模式) 与 M 模式 (机器模式) 两种权限级别。通过在 CPU 内部维护一个 2bit 的寄存器用于记录

当前所处的权限级别，CPU 可以基于此判断操作 (如 zicsr 指令) 在当前权级下是否合法，如不合法则抛出异常，交由中断控制硬件与服务程序进行处理。

5.2.3 中断控制硬件

在上述架构的基础上，我们参考官方文档实现了基本的中断控制硬件系统，并在此基础上实现了对 ecall，ebreak，mret 三条指令的中断硬件支持。该系统所支持的中断信号列表如图15

中断/异常名	源自模块	mcause(自判)	mtval	备注
外部中断	PLIC	1/11		
Instruction Access Misaligned	Memory Dispatch	0/0	PC	阶段: IF
Instruction Access Fault	Memory Dispatch	0/1	PC	IF
Illegal Instructions	ID	0/2	Instruction	ID
Breakpoint (ebreak)	ID	0/3	0	ID
load address misaligned	Memory Dispatch	0/4	address	MEM
load address fault	Memory Dispatch	0/5	address	MEM
store misaligned	Memory Dispatch	0/6	address	MEM
store fault	Memory Dispatch	0/7	address	MEM
Ecall from M (ecall)	ID	0/11	0	ID
mret	ID	NA	NA	ID 注意：状态为：回子程序状态

图 15: 中断类型示意图

上述中断信号将分别由对应的模块探测并生成对应的中断信号与 mtval，随后该

信号传递至中断控制硬件模块进行进一步的处理。对于除 mret 中断以外的其它中断，中断控制硬件将会执行如下操作：1. 将 mepc 寄存器置为当前 PC 寄存器中的值。这一步是为了记录中断发生的指令地址，以供中断服务程序处理完毕后返回

2. 将 PC 置为 mtvec 寄存器所存储的值。这一步实现了从源程序切换到中断服务程序进行执行

3. 生成中断信号对应的信号写入 mcause 与 mtval。该寄存器的信息将供中断服务程序判断当前的中断类型等信息

4. 将 mstatus.MIE 写入 MPIE，随后将 MIE 置为 0。该操作将中断发生前的外部中断启用状态 (MIE) 存入 MPIE 中，并在中断服务程序一开始暂时屏蔽外部中断。

5. 将 mstatus.MPP 置为当前权限等级，并将当前权限等级置为 Machine 等级。这一步记录了中断发生前的权限等级。由于中断服务程序需要 Machine 级别的权限以对 CSR 寄存器进行访问操作，因此还需要将当前权限等级置为 Machine 级别

该过程的关键 chisel 代码如图16

```
io.fault_occurs := true.B
mepc := io.pc
io.fault_new_PC := mtvec
mcause := _mcause
mtval := _mtval
mstatus := mstatus.bitSet(3.U, false.B) //MIE禁用中断写为0
//    read_MPIE := read_MIE //MPIE <-MIE
mstatus := mstatus.bitSet(7.U, read_MIE)
//    read_MPP := cur_privilege
mstatus := Cat(mstatus(31, 13), cur_privilege, mstatus(10, 0))
cur_privilege := PrivilegeType.Machine // 切换至M模式
```

图 16: 中断切换关键代码

而对于 mret 指令，该指令需要支持从中断服务程序中切换回源程序继续执行，因此需要执行与上述操作对应的相反操作，具体操作为：1. 将 PC 置为 mepc

2. 将 mstatus.MIE 置为 MPIE 段的值。这一步恢复了中断发生前的外部中断控制信号

3. 将当前权限等级置为 mstatus.MPP 的值。这一步恢复了中断发生前的权限等

级

通过上述流程，该系统实现了完整的对中断的捕获，并将具体的处理细节交由中断服务程序进行处理。中断服务程序可以自定义对不同类型中断的具体处理方式，实现了中断捕获（硬件）与中断处理（软件）之间的解耦。

5.2.4 中断服务程序，ecall 与 ebreak 的实现

在上一章节所描述的中断控制体系中，ecall，ebreak 等指令将与其它指令一同被视为内部异常的发生，中断服务程序将依据 mcause 寄存器中所存储的中断类型判断当前触发的是何种中断，并交由对应的代码段进行进一步处理。我们所实现的 ecall 与 ebreak 功能列表如图**??**所示 对于中断服务程序的关键代码，当中断触发时，CPU

ecall

a7	load	to
1	switch	a0
2	a0	7seg
3	a0	led
4	exit	---

图 17: ecall 功能列表

将会切换到其基地址开始执行，首先，中断服务程序需要暂存原寄存器中的值，并依

ebreak

button	function
b3	confirm reg
b4	go back

图 18: ebreak 功能列表

据 mcause 寄存器判断中断发生的实际类型，随后跳转到对应的代码块进行进一步执行，如代码图19所示

```
addi t0, zero, 1
beq a7, t0, sys_ecall_case1
addi t0, t0, 1
beq a7, t0, sys_ecall_case2
addi t0, t0, 1
beq a7, t0, sys_ecall_case3
addi t0, t0, 1
beq a7, t0, sys_ecall_case4
# if no cases satisfy, then output error
addi t0, zero, 1
slli t0, t0, 22
sw t0, (a1)
jal sys_ecall_goBack
```

图 19: 中断服务程序

而对于 ecall 中断，该中断代码将依据功能需求选择轮询 MMIO 地址或将寄存

器值写入 MMIO 地址，随后当操作完成时返回子程序继续运行，其部分代码模块如图20所示

```
sys_ecall_case2:
    lw t0, (a2)
    andi t0, t0, 4
    beq t0, zero, sys_ecall_case2
    sw a0, (a4)
    jal sys_ecall_goBack
```

图 20: ecall 处理示例

最后在跳回过程中，中断服务程序将恢复原寄存器的值，并将跳转地址置为中断触发源地址 +4（以避免陷入中断指令中），随后跳回原程序继续执行，如代码图21所示

通过上述操作，我们完整实现了对于系统中断的处理与 ecall，ebreak 指令的支持

5.3 Vga 实现说明

vga 设计参考于 [3]。vga 模块实现 25MHz 分频，接受 32bit 二进制输入，通过硬件连接实现向 8 位十六进制数的转换，并将其呈现在显示屏上。显示器从屏幕左上角开始，从左向右逐点扫描，每扫描完一行，电子束回到屏幕下一行左边起始位置开始扫描。通过行同步、场同步、消隐区的控制实现上述过程。对于扫描到的每个像素点，通过对于输出 rgb 信号的调控，实现对于红绿蓝三种颜色电压的控制以呈现出不同的颜色。在 vga 代码中，对显示图像进行了 2048bit 的二进制编码，使 32×64 区域内能够显示对应的图像。在对输入的 32bit 二进制数进行解构后，采用 MUX 结构从预设定好的候选值中选定对应的 2048bit 数，控制像素点的颜色。

```
sys_ecall_goBack:
    lw t0, (a2)
    andi t0, t0, 4
    bne t0, zero, sys_ecall_goBack
    csrrs t0, mepc, zero
    addi t0, t0, 4
    csrrw zero, mepc, t0
    lw a1, 4(sp)
    lw a2, 8(sp)
    lw a3, 12(sp)
    lw a4, 16(sp)
    lw t0, 20(sp)
    addi sp, sp, 24
    mret
```

图 21: 中断服务程序推出

5.4 自创工具链实现说明

由于我们小组均使用 Windows 系统进行开发，因此我们在开发环境中部署了 riscv-none-elf 编译链 [5]，并在此基础上编写了批处理脚本。该脚本支持从 C 代码或汇编代码编译生成 elf 文件，导出文件的 16 进制机器码文本，端序转换，coe 文件生成与 riscv 机器码反汇编等功能，该系统的示意如图22

对于 C 代码文件的编译，其编译过程将包括预处理（生成.i 文件），编译（生成.s 汇编代码），汇编（生成.o 二进制文件），链接（生成 elf 可执行文件）四个步骤。在 riscv-none-elf 中，上述步骤将对应如图23所示的四条指令

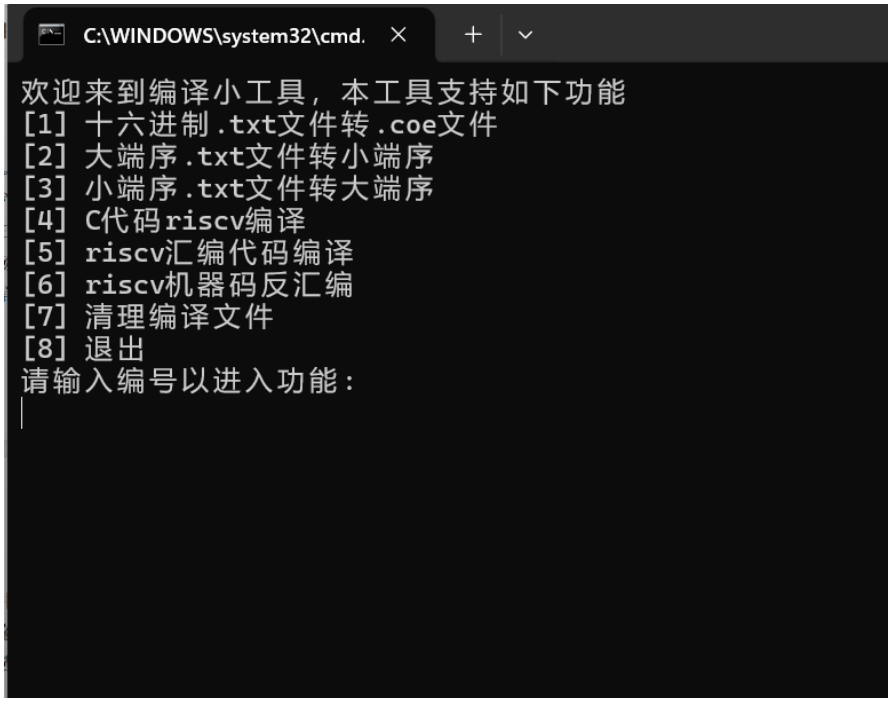


图 22: 编译工具链示意图

```
:: 预处理
riscv-none-elf-gcc -E %fileName% -o %fileName:.c=.i%
:: 编译
riscv-none-elf-gcc -S -march=rv32i %fileName:.c=.i% -o %fileName:.c=.s%
echo 汇编代码输出至：%fileName:.c=.s%
pause
:: 汇编
riscv-none-elf-as -march=rv32i -o %fileName:.c=.o% %fileName:.c=.s%
:: 链接
riscv-none-elf-ld -nostdlib -T linker_script.ld -o %fileName:.c=.elf% %fileName:.c=.o%
```

图 23: Enter Caption

在第四阶段，由于内存空间段的分布不同，需要配置 linker 文件指定如数据段，代码段等各段空间的。依据本项目所设计的 CPU 内存分布定义我们编写了相应的链接配置文件，其部分代码如图24

```
MEMORY
{
  TEXT (rx) : ORIGIN = 0x00000000, LENGTH = 0x0000FFFF
  DATA (rw) : ORIGIN = 0x00010000, LENGTH = 0x0000FFFF
}

SECTIONS
{
  .text :
  {
    *(.text)
    . = ALIGN(4);
    PROVIDE(   global pointer$ = . + 0x10200);
  } > TEXT

  .data :
  {
    *(.data)
    . = ALIGN(4);
  } > DATA
```

图 24: 链接配置文件

数据导出的部分使用了 hexdump 实现将可执行代码导出至十六进制文本格式，而对于其它功能则通过编写了 python 脚本与使用编译链指令进行实现。最后，上述代码文件将由 main 批处理文件依据用户输入进行调用。

5.5 流水线实现说明

5.5.1 整体设计

本项目整理采用了经典了五级流水架构实现对流水功能的支持。在经典的五级流水架构中，指令的执行过程将分为取指、译码、执行、访存、写回五个步骤。在实际实现中，我们在各个层级插入 IF-ID,ID-EX,EX-MEM,MEM-WB 四个夹层寄存器以暂存各步骤的执行结果，并在随后将执行结果递送给下一阶段进行执行。此外，各夹层之间存储的执行结果还将交由冲突控制模块进行统一的分析，并由冲突控制模块决定各夹层之间是否进行暂停（夹层寄存器信号保持不变）或冲刷（将夹层寄存器信号

置为空指令信号）操作。该设计的具体架构示意图参见前文的 CPU 内部结构图章节。

5.5.2 数据前递

对于在流水线状态下相邻指令之间可能存在的数据冒险，本架构采用了数据前递的方式实现对数据冒险的处理。在具体实现中，ID-EX,EX-MEM 阶段分别拥有寄存器用于指示当前指令将要写回的寄存器编号（0 代表不写回），而冲突控制模块将探测 ID 阶段指令译码后的读取地址是否与 ID-EX,EX-MEM 两层中的目标写回寄存器一致，如一致则发生了数据冒险，冲突控制模块将进一步探测冲突指令生成结果的阶段，若冲突发生阶段已经生成了结果，则可以直接前递结果，否则（如 Load-Use 型冲突）则将暂停 PC 寄存器与 IFID 夹层寄存器的更新，并在 ID-EX 层执行冲刷操作（插入空指令）以等待结果生成。上述过程的冲突控制模块代码参见图25

```
//两个rs1 use,前递EX的
when(io.IDEX_write_reg
  && io.IDEX_reg_to_write =/= 0.U
  && io.rs1_use === io.IDEX_reg_to_write) {
  when(io.IDEX_reg_result_stage === ResultStageType.EX.getUInt) { //检查结果的EX得到还是MEM得到
    assign_output(
      rs1_forward_type = ForwardType.EXForward.getUInt
    )
  }.otherwise { //MEM
    assign_output(
      nextpc_control_signal = NextPCControlSignal.Stall.getUInt,
      IDEX_control_signal = LayerControlSignal.NOP.getUInt,
      IFID_control_signal = LayerControlSignal.Stall.getUInt,
    )
  }
}.elsewhen(io.EXMEM_write_reg
  && io.EXMEM_reg_to_write =/= 0.U
  && io.rs1_use === io.EXMEM_reg_to_write) { //直接前递即可
  assign_output(
    rs1_forward_type = ForwardType.MEMForward.getUInt
  )
}
```

图 25: 数据冲突部分代码

5.5.3 指令预测

对于 B 类型等分支跳转指令所存在的控制冒险，本架构使用分支预测与指令冲刷的方式进行处理。分支预测过程主要采用了基本的多状态动态分支预测。其具体代码实现如下 26

```
//prediction module
when(io.instruction(6, 0) === "b1100011".U) {
  val imm = Cat(Fill(20, io.instruction(31, 31)),
    io.instruction(31, 31), io.instruction(7, 7),
    io.instruction(30, 25), io.instruction(11, 8))
  cache.io.read_en := true.B
  when(cache.io.dataOut <= min_weak_jump.U) { //jumping
    io.predict_new_pc := io.pc + (imm << 1)
  }.otherwise {
    io.predict_new_pc := io.pc + 4.U
  }
}
//updating module
when(prediction_result_get) {
  when(io.prediction_failure.failure_occurs) {
    when(cache.io.write_data_out <= min_weak_jump.U) { //predict jump but failed
      cache.io.dataIn := cache.io.write_data_out + 1.U
    }.otherwise {
      cache.io.dataIn := cache.io.write_data_out - 1.U
    }
  }.otherwise { //success
      printf("cur wdout: %d\n",cache.io.write_data_out)
    when(cache.io.write_data_out <= min_weak_jump.U) { //predict jump and success
      when(cache.io.write_data_out > 0.U) {
        cache.io.dataIn := cache.io.write_data_out - 1.U
      }
    }.otherwise {
      when(cache.io.write_data_out < max_strong_no_jump.U) {
        cache.io.dataIn := cache.io.write_data_out + 1.U
      }
    }
  }
```

图 26: 分支预测

```
is(BranchType.BType.getUInt) {
  val correct_pc = Mux(io.cmp_result,io.pc + io.imm,io.pc+4.U)
  when(io.predict_next_pc =/= correct_pc) { //prediction failed
    io.prediction_failure.setFailed(io.pc)
    control_hazard:=true.B
    io.new_pc := correct_pc
    assign_output(
      nextpc_control_signal = NextPCControlSignal.NewPC.getUInt,
      IFID_control_signal = LayerControlSignal.NOP.getUInt,
      IDEX_control_signal = LayerControlSignal.NOP.getUInt
    )
  }.otherwise{
    io.prediction_failure.setSuccess(io.pc)
  }
  if(GenConfig.s.logPrediction){
    Logger.logPrediction(io.pc,io.predict_next_pc,io.predict_next_pc === correct_pc)
  }
}
```

图 27: 控制冒险处理

对于一条 B 类型指令，当 CPU 在取指阶段发现所取指令为该类型时，其将会将该指令存储至 CPU 内部的一个全相联分支指令缓存（分支历史表）中，，并记录初始状态为 week-jump 状态，并将 PC 更新为跳转后的 PC 地址。随后当该指令执行至 EX 阶段时，冲突控制模块将会比较该指令的实际跳转地址与 ID 阶段（即预测的指令）地址是否一致，并将结果传递给 PC 更新模块，PC 更新模块将依据该指令的预测结果更新预测状态机，如当 week-jump 状态预测失败时，状态将会切换至 week-not-jump，反之测切换至 strong-jump，同时将 PC 置为实际的跳转 PC 地址。同时，若分支预测失败，冲突控制模块将会将 IF-ID,ID-EX 层执行冲刷操作，上述流程实现了完整的分支预测与控制冒险处理。此外，在具体的实现中，上述代码使用了参数化的代码方式，从而支持了任意位数的 BHT 分支预测，参数化的设计方式提高了系统的可调节能力。

6 测试与结果

由于 CPU 的硬件核心部分使用 chisel 编写，我们直接使用 chisel 在软件上进行仿真测试以解决当前 CPU 存在的逻辑性漏洞，事实表明，开发板上的表现与在软件上的表现并没有显著差异。单个测试用例示意图如下：

```
it should "beq1" in {
  //    addi x1,x0,1
  //    addi x2,x0,1
  //    beq x2,x1,NEXT
  //    addi x3,x0,1
  //    addi x4,x0,1
  //    addi x5,x0,1
  //    NEXT:
  //      addi x3,x0,10
  //    -> x3=10,x4=0,x5=0

  load_instructions("jump1.txt")
  test(new CoreTop) { cpu =>
    run_instructions(cpu, 7)
    checkRegsInCPU(cpu, 3, 10.U)
    checkRegsInCPU(cpu, 4, 0.U)
    checkRegsInCPU(cpu, 5, 0.U)
  }
}
```

图 28: 测试示意图

我们所有的测试如表所示:

测试用例	测试内容	测试情况
CoreTopTest	基础指令集	passed 30 of 30
TotalTopTest	外设交互	passed 17 of 17
software1Test	汇编场景 1 测试	passed 19 of 19
software2Test	汇编场景 2 测试	passed 52 of 52
CoreTopCSRTest	两模式特权架构, 中断控制硬件,ecall 与 ebreak 测试	passed 4 of 4
otherTest	Pipeline 冲突测试	passed 4 of 4

表 3: 测试用例说明

6.1 ISA 指令集测试 (包括 lui 与 auipc)

对于 ISA 指令集，我们对每一条指令都编写了 1 到多条测试，总共接近 50 条测试。测试结果如下：

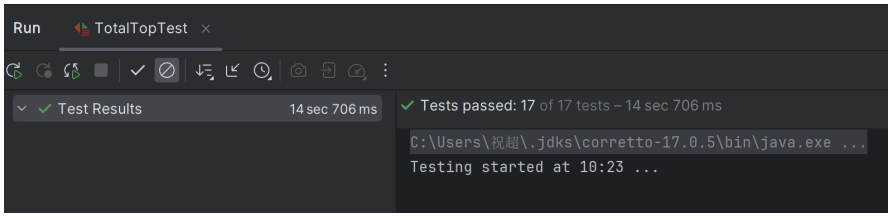


图 29: totalTopTest 结果

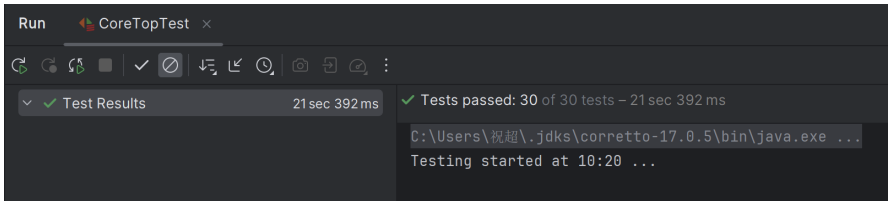


图 30: coreTopTest 结果

6.2 基本汇编场景测试

我们对每个 case 都写了一条及以上的测试，尽可能地涵盖了所有可能发生的情况，并且在开发板上的测试表现几乎与仿真一致。

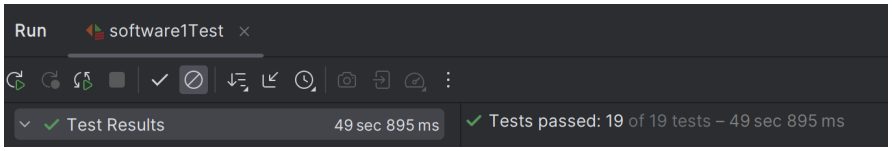


图 31: software1Test 结果

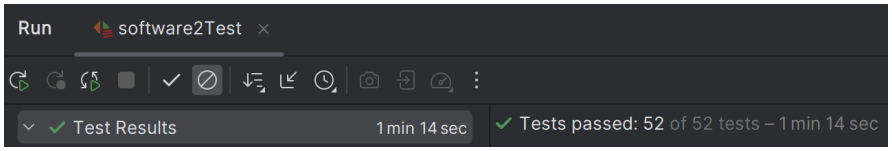


图 32: software2Test 结果

6.3 两模式特权架构, 中断控制硬件,ecall 与 ebreak 测试

在 CSR 指令相关的测试中，除了通过运行前面的测试以确保 CSR 指令与正常指令没有冲突之外，我们还针对 CSR 指令的使用场景设计了一些测试用例，如基本 CSR 指令，非法 CSR 指令（特权架构），ecall，ebreak，非法地址访问等进行了测试，测试结果如下：

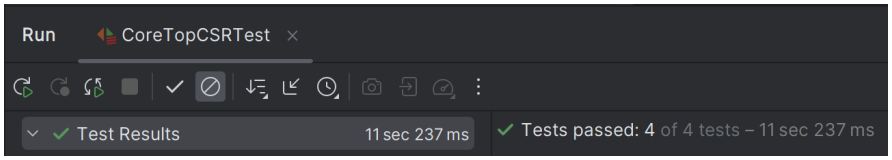


图 33: CoreTopCSRTestTest 结果

6.4 Pipeline 冲突测试

在流水线相关的测试中，除了通过运行前面的测试以确保流水线 CPU 与正常指令没有冲突之外，我们还针对流水线的使用场景设计了一些测试用例，例如数据冒险，控制冒险相关的测试，测试结果如下：

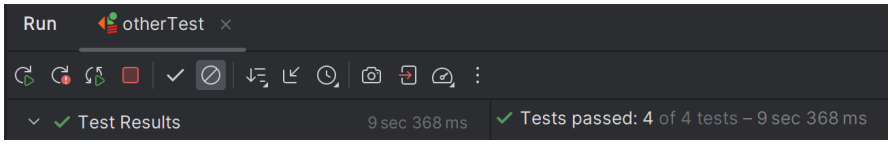


图 34: otherTest 结果

7 问题与总结

7.1 复位的重要性

一开始我们没有绑定 reset 的引脚，直接导致硬件开发板的初始情况不可知，于是硬件开发板上出现了一些软件测试中从来没有出现过的情况，我们认为可能是 PC 的值出现了问题，后面在绑定了 reset 之后，硬件上的表现就和软件仿真一样了。另外，在 reset 的时候，我们采取的是同步 reset 的策略，然而一开始我们对 ip 核中调用的时钟也绑上了 reset，就导致我们的 reset 一开始并未达到预期的作用。总结来说，就是要绑定 reset，并且要注意同步 reset 和异步 reset 的区别，并且在同步 reset 的时候不要给时钟绑定 reset。

7.2 不要盲从 chisel

我们花了比较多时间解决的一个问题是在 uart 的通信上面，然而，我们层层排查到最后，发现是由于 chisel 的代码不够规范而导致最后生成的引脚少了一个，由于 chisel 没有报错（或者警告），我们很晚才查到生成的 verilog 代码并发现这个问题。这件事情也提醒我们：有一些问题 chisel 是不会给你指出来的，所以如果遇到了与软件不符合的情况，也要尽快查看一下 verilog 代码是否有严重的“低级失误”。

7.3 多做测试

我们的开发顺序是：实现单周期 CPU 到实现 CPU 特权架构再到实现流水线的，那么我们在进行后面的开发的时候如何保证不影响到前面呢？这就是我们的测试在发挥作用了，对于每一个新加入的功能，我们都要求需要跑通前面的测试，然后再对应新加入的功能进行相应的测试，因此，我们几乎能保证每一个新加入的功能没有逻辑性的 bug。并且，得益于 github 良好的版本控制以及分支管理体系，我们能够同时进行不同模块的开发，这大大提升了我们的效率。

参考文献

[1] Andrew Waterman and Yunsup Lee and Rimas Avizienis and David Patterson and Krste Asanović. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Document Version 20191213*. RISC-V Foundation, December 2019.

[2] Andrew Waterman and Yunsup Lee and Rimas Avizienis and David Patterson and Krste Asanović. *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 20190608*. RISC-V Foundation, June 2019.

[3] CSDN 用户: 咖啡 0 糖. *FPGA—VGA 显示器显示彩条*.

[4] GithubUser: Alex Forencich. *AXI4-Stream Opensource Verilog UART*.

[5] Github Community: xPack 3rd Party Development Tools. *riscv-none-elf-gcc-xpack*.